1. Project Title & Overview

Project Name: SkillMint

One-line Description: A premium, student-focused freelance marketplace designed to bridge the gap between academic skills and professional opportunities.

Detailed Explanation: SkillMint is a high-performance, full-stack web application that allows students to monetize their university skills (e.g., coding, design, writing) by creating and managing service listings (Gigs). It provides a structured environment where clients (Buyers) can browse available services, post custom requests, and interact with students through a real-time messaging system. The platform integrates secure authentication, complex relational data management, and instant communication to ensure a professional user experience.

Real-world Use Case: A university student proficient in React can list a "Frontend Development" gig on SkillMint. A local startup looking for affordable talent can find this student, chat in real-time to discuss requirements, place an order, and track the project status until completion—all within a single, secure environment.

2. Tech Stack

Frontend

- **React (Vite):** Chosen for its component-based architecture and fast development cycles via Vite's HMR (Hot Module Replacement).
- **Tailwind CSS:** Used for rapid, utility-first styling to achieve a premium, custom UI without the overhead of heavy CSS frameworks.
- **Framer Motion:** Implemented for high-quality micro-animations and smooth page transitions to enhance the user experience.
- **Lucide React:** A consistent set of icons used for clear, standard UI representations.

Backend

- **Node.js & Express.js:** Provides a scalable and lightweight environment for handling RESTful API requests and middleware logic.
- **Socket.io:** Enables real-time, bi-directional communication between client and server for instant messaging features.

Database & ORM

- **Supabase (PostgreSQL):** A cloud-hosted relational database that provides robustness, scalability, and built-in features like real-time listeners.
- **Prisma ORM:** Used for type-safe database access, simplified schema migrations, and intuitive relationship management between models.

Authentication & Deployment

- **Google OAuth:** Integrated for secure, one-click user authentication.
- **Vercel:** Hosting platform for the frontend, optimized for React applications with built-in CDN and CI/CD.
- **Render:** Hosting platform for the backend server, supporting persistent Node.js processes and WebSockets.

3. Folder Structure

```
/SkillMint (Root)
├── backend/                    # The server-side logic and database management
│   ├── middleware/             # Authorization and request-validation logic
│   ├── prisma/                 # Database schema definitions and migrations
│   ├── routes/                 # Individual API endpoints (Auth, Gigs, Orders,
etc.)
│   ├── utils/                  # Helper functions and data formatters
│   ├── .env                    # Server-side environment variables
│   ├── index.js                # Main entry point (Express and Socket.io setup)
│   └── package.json            # Backend dependency and script management
├── frontend/                   # The client-side React application
│   ├── public/                 # Static assets (images, icons)
│   ├── src/                    # Main source code
│   │   ├── components/         # Reusable UI elements (Navbar, Footer, etc.)
│   │   ├── context/            # Global state management (Auth and Theme)
│   │   ├── pages/              # Main view components (Marketplace, Dashboards)
│   │   ├── App.jsx             # Main application component and routing
│   │   └── main.jsx            # React entry point
│   ├── vercel.json             # Vercel-specific deployment configuration
│   ├── vite.config.js          # Vite build and development configuration
│   └── package.json            # Frontend dependency and script management
├── package.json                # Root package.json for npm workspaces (Monorepo)
└── README.md                   # Documentation for the project
```

4. Core Code Explanation

Database Schema (backend/prisma/schema.prisma)

The schema defines the relational structure of the application. Relationships are critical for connecting Gigs to Users and Orders to Gigs.

```
model User {
  id        String   @id @default(uuid())
  email     String   @unique
  name      String?
  gigs      Gig[]    // Relationship: One user can have many service listings
  orders    Order[]  // Relationship: One user can place many orders
}

model Gig {
  id          String   @id @default(uuid())
  title       String
  description String
  price       Float
  userId      String
  user        User     @relation(fields: [userId], references: [id]) // Connects Gig
```

```
to Seller
  orders      Order[]
}
```

Real-time Communication (backend/index.js)

Socket.io is initialized alongside Express to handle bi-directional events.

```
// Setup Socket.io for real-time chat
const io = new Server(server, {
  cors: { origin: process.env.CLIENT_URL } // Restrict access to authorized frontend
});

io.on('connection', (socket) => {
  // Join a specific room based on Order ID for private communication
  socket.on('join_room', (roomId) => {
    socket.join(roomId);
  });

  // Handle message sending and broadcast to the room
  socket.on('send_message', (data) => {
    io.to(data.roomId).emit('receive_message', data);
  });
});
```

Authentication Context (frontend/src/context/AuthContext.jsx)

State management ensures that user details are available throughout the application.

```
export const AuthProvider = ({ children }) => {
  const [user, setUser] = useState(null);

  // Check for existing token and validate with backend on mount
  useEffect(() => {
    const token = localStorage.getItem('token');
    if (token) validateToken(token);
  }, []);

  return (
    <AuthContext.Provider value={{ user, setUser }}>
      {children}
    </AuthContext.Provider>
  );
};
```

5. Application Flow

1. **Request Initiation:** A user interacts with the React frontend (e.g., clicking 'Create Gig').
2. **API Call:** The frontend sends an HTTP POST request to the Node.js backend with a JWT token for authorization.

3. **Middleware Validation:** The backend verifyToken middleware checks the JWT to ensure the user is authenticated.
4. **Business Logic:** The `/api/gigs` route receives the request, processes the data, and interacts with Prisma.
5. **Database Interaction:** Prisma ORM performs an SQL query on the Supabase PostgreSQL database to save the record.
6. **Response:** The backend sends a JSON response back to the frontend.
7. **UI Update:** React updates the state and the user sees the newly created gig on their dashboard.

6. Database Design

Key Entities:

- **User:** Stores profile information and unique identifiers.
- **Gig:** Contains service details, pricing, and a foreign key to the User (Seller).
- **Order:** Tracks the transaction between a Buyer and a Gig. It links to both the Service and the Participants.
- **Message:** Stores individual chat entries with timestamps and room identifiers for thread organization.

Relationships:

- **One-to-Many (User-Gig):** A student can offer multiple services.
- **Many-to-One (Order-Gig):** Multiple clients can buy the same gig.
- **Many-to-One (Message-Order):** A single order/transaction contains many messages between participants.

7. API Endpoints

Method	Endpoint	Input	Purpose
POST	`/api/auth/google`	Google ID Token	Handles Google OAuth login and user creation.
GET	`/api/gigs`	None	Fetches all available services for the marketplace.
POST	`/api/gigs`	Gig Object	Creates a new service listing (Authenticated).
GET	`/api/gigs/:id`	UUID	Fetches detailed information for a specific gig.
POST	`/api/orders`	Gig ID, Buyer ID	Creates a new project/transaction.
GET	`/api/messages/:roomId`	String (Room ID)	Retrieves the chat history for a specific order.

8. Authentication

Flow:

1. **Frontend Entry:** User clicks "Sign in with Google".
2. **Google Approval:** Browser redirects to Google; user approves.
3. **Token Exchange:** Vercel receives a credential token and sends it to the Render backend.
4. **JWT Issuance:** Backend validates the Google token, creates/updates the user, and issues a local JSON Web Token (JWT).
5. **Session Persistence:** The JWT is stored in `localStorage` on the frontend for subsequent API requests.

9. Key Features Breakdown

Dynamic Gig Marketplace

The marketplace uses React's state hooks to filter and search through gigs without page reloads. Prisma ensures that only active and valid gigs are retrieved from PostgreSQL.

Real-time Messaging System

Built on Socket.io, it uses distinct "rooms" based on Order IDs to isolate conversations. This ensures that only the authorized buyer and seller can see their project discussions.

Multi-step Gig Creation Flow

An intuitive multi-step form on the frontend guides students through title, description, pricing, and category selection, ensuring data completeness before reaching the backend.

10. Error Handling

Backend:

A centralized error-handling middleware captures all try/catch failures.

- **Input Validation:** Backend checks for required fields and sends 400 Bad Request if data is missing.
- **Database Failures:** Prisma's error codes are intercepted to send user-friendly 500 Internal Server Error messages if the database is unreachable.

Frontend:

- **Axios Interceptors:** Detects 401 Unauthorized responses to automatically log the user out if their session expires.
- **Loading/Error States:** UI components provide visual feedback (spinners or alert boxes) when background processes fail.

11. Deployment Summary

Frontend (Vercel)

- **Framework:** Vite/React
- **Root Directory:** `/frontend`
- **Configuration:** `vercel.json` used for handling client-side routing rewrites.

Backend (Render)

- **Runtime:** Node.js

- **Service Type:** Web Service (with persistent runtime)
- **Root Directory:** `/backend`
- **Build Command:** `npm install && npx prisma generate` (Ensures the database client is ready before startup).

12. Interview-Focused Section

Q: Why did you choose PostgreSQL over MongoDB for a marketplace?

A: Marketplaces rely on strong consistency and relational integrity. PostgreSQL (via Supabase) allows for complex joins between Gigs, Orders, and Users, ensuring that an order cannot exist without a valid user and gig. MongoDB's eventual consistency wouldn't have been ideal for tracking payments or transaction statuses.

Q: How do you handle real-time synchronization across different users?

A: I implemented Socket.io to establish a persistent connection. When a message is sent, the server identifies the Room ID and broadcasts the data only to users in that room. This minimizes server load and ensures that data is only pushed to those who need it.

Q: What was the biggest challenge during deployment?

A: The biggest challenge was managing the monorepo structure on two different platforms. I had to specifically configure the "Root Directory" on both Vercel and Render to ensure they only built the relevant part of the codebase while still recognizing the root-level dependencies.

13. Key Learnings

1. **ORMs (Prisma):** Learned how to manage relational schemas and perform type-safe queries, which reduced runtime bugs compared to raw SQL.
2. **State Synchronization:** Gained experience in keeping the frontend UI in sync with a remote database and real-time socket events.
3. **Cross-Origin Communication:** Mastered CORS (Cross-Origin Resource Sharing) configurations to allow a frontend hosted on Vercel to securely talk to a backend hosted on Render.
4. **DevOps Basics:** Understanding how to configure build pipelines and environment variables for a professional production deployment.